Constitutional Storage

Governed Occupancy, Governed Absence, and the Receipted Transitions Between Them

A system is not full when space runs out. It is full when its state is ungoverned.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Modern storage systems already manage free space through free lists, allocators, compaction, tiering, garbage collection, replication, quotas, snapshots, reserves, and recovery logs. This paper does not claim to invent those mechanisms. Its central claim is different: *storage health depends on constitutional consistency across occupied state, free state, movement authority, deletion semantics, repair obligations, and emergency reserves — and these mechanisms are usually specified as independent operational subsystems rather than as one coherent admission, receipt, repair, and authority model.* Constitutional Storage offers that integrated specification.

Dense storage systems often inherit a *false unity* of a different kind. They collapse object identity, object placement, free-space discovery, lookup routing, deletion semantics, fragmentation repair, auditability, and execution authority into a single operational surface. In hash tables this collapse appears in the popular "full parking lot" problem: as a table approaches capacity, empty slots become rare, greedy insertion forms clusters, and lookup or insertion appears to slow under the weight of probability. In allocators, file systems, object stores, and artifact archives, the same collapse appears as fragmentation, stale descriptors, free-space table growth, expensive compaction, ambiguous deletion, and recovery failure. But the deeper failure is upstream: when occupied state is not constitutionally governed — when retention is unjustified, movement authority is unclear, deletion semantics are ambiguous, repair debt is unbounded, and reserve discipline is informal — free-space failure is the symptom rather than the disease.

This paper proposes **Constitutional Storage**: a storage architecture in which objects, spaces, transitions, receipts, repair obligations, low-overhead exceptions, and capacity reserves are separately represented and separately governed. The architecture treats *occupancy* and *absence* as parallel constitutional states, joined

by *admissible transitions* between them. It introduces a Things table for object descriptors, a Spaces table for free extents, a transition constitution for admissible mutations, receipts for replayable legitimacy, and jurisdictional capacity classes for the differentiated reserves modern systems already use informally.

Motivated by the popular account of Andrew Krapivin's work on Elastic Hashing and the long shadow of Andrew Yao's dense-hashing frame, this paper treats the "firebreak" insight as a special case of a broader principle: empty space is not nothing. It is state. Where open addressing manages empty slots inside the lookup geometry of a hash table, Constitutional Storage separates object lookup from space allocation through independently governed Things and Spaces tables. It then governs the transitions between them.

A worked parking-infrastructure analogy in Section 2.5 illustrates the same separation between object lookup, free-space visibility, and jurisdictional routing — and the limits of physical analogies for digital occupancy governance. The intellectual contribution of the present paper is not the discovery of the pattern but its reconstruction as a unified constitutional specification spanning occupancy, absence, movement, repair, authority, and receipts.

The model further introduces three distinct cost-governance mechanisms. **Repair governance** controls physical disorder: fragmentation, compaction, pinned objects, and free-space table growth. **Batch governance** controls receipt overhead by grouping many small operations into verifiable aggregate transitions. **Fast-lane governance** controls semantic significance by allowing trivial operations to use compact primitives only while they remain within scoped authority, bounded debt, and reconciliation rules.

The result is not a claim that metadata makes physical costs vanish. Moving bytes still costs time. Compaction still amplifies writes. Dense indexes still matter. Receipts still consume space. Rather, Constitutional Storage claims that these costs become legible, governable, schedulable, and accountable once storage state is separated into its constitutional axes. The full parking lot is not abolished by magic. It is dissolved as sovereign architecture and reduced to one licensed province inside a larger governed system.

---

1. Introduction

Storage systems are often designed as if emptiness were obvious.

An empty slot is simply empty.

A free block is simply free.

A deleted file simply returns space.

A hash table slot either contains a key or does not.

This simplicity is useful at small scale. It becomes unstable at density.

As systems approach fullness, absence stops behaving like nothing. It becomes structure. It has shape, locality, cost, history, ownership, and consequence. One free byte beside another free byte is not the same constitutional fact as one free byte separated from another by an immovable artifact. Ten thousand scattered holes are not equivalent to one contiguous open field. A deleted range is not merely missing content; it is a transition in object identity, placement, history, and future admissibility.

The popular story of dense hash tables gives this instability a memorable image: the full parking lot. When the lot is mostly empty, a driver finds a space quickly. When the lot is almost full, the driver circles. Greedy strategies make the problem worse by taking the first available space, producing clusters that future searches must cross. Elastic Hashing changes the story by intentionally skipping some available spaces. Those empty spaces become firebreaks. They prevent clusters from merging and preserve future navigability.

This paper accepts the firebreak lesson and lifts it into a broader storage architecture.

The important discovery is not only that an algorithm may skip a slot.

The important discovery is that absence has structure.

An empty location may be:

```
a firebreak
a free extent
a compaction target
a deferred repair obligation
a fast-lane budget
a reserved jurisdiction
a fragmentation symptom
a constitutional violation
```

A storage system fails when it treats all of these as the same kind of nothing.

Constitutional Storage therefore begins from a separation:

```
object identity != object placement
object placement != free-space state
free-space state != mutation admissibility
mutation admissibility != historical receipt
historical receipt != current authority
```

When these are collapsed, dense storage becomes chaotic.

When they are separated, dense storage becomes governable.

---

2. Scope

Before scoping the paper, one boundary needs to be set explicitly. Constitutional Storage does not argue that existing systems lack free-space management. They do not. Modern storage systems already use free lists, allocation classes, compaction, garbage collection, tiering, replication, quotas, snapshots, reserves, and recovery logs. The claim is different: these mechanisms are often specified as operational subsystems rather than as one constitutional model of admissible occupancy, absence, movement, repair, and authority. Constitutional Storage asks not only where space is free, but why occupied space remains legitimate, when it may move, when it must be deleted, when disorder becomes debt, and when emergency reserves may be consumed. The paper is a requirements framework for integrated storage governance, not a proposal for new low-level mechanisms.

Constitutional Storage does not claim to eliminate physical cost.

It does not make compaction free. It does not make indexes unnecessary. It does not make receipt overhead disappear. It does not replace dense hash tables, B-trees, LSM trees, slab allocators, or cache-local primitives.

Its claim is narrower and stronger:

> Allocation, identity, repair, authority, and historical legibility should be separately represented and separately governed.

The paper addresses storage systems in which objects may have durable identity, semantic meaning, authority constraints, audit requirements, deletion semantics, or repair obligations. The model is less useful for trivial, isolated, short-lived data structures where full governance would cost more than it clarifies. For those cases, the model introduces a governed fast lane rather than pretending every byte deserves full ceremony.

Constitutional Storage is therefore not a universal replacement for compact data structures.

It is a constitutional architecture for deciding when compactness is sufficient, when governance is required, and how the two coexist.

2.5 A Worked Example: Mall Parking as Constitutional Storage

A familiar physical-infrastructure system has been implementing the architecture this paper proposes for decades. Large shopping-mall parking structures, particularly in regions where indicator-light systems have been deployed, instantiate the core Constitutional Storage pattern in concrete and humanly observable form. The worked example below maps each component of the parking system to a paper concept, identifies what the parking solution gets right that conventional dense-storage designs get wrong, and notes the gap that allowed the pattern to remain unrecognized in the systems-theory literature.

The parking infrastructure. A representative mall parking structure has the following observable components: each parking space has an overhead indicator light (typically green for free, red for occupied), the lights are driven by a sensor-based detection layer that tracks per-space occupancy state, each floor displays an aggregate sign showing the count of free spaces on that floor, spaces are numbered for identification, and drivers make routing and parking decisions by reading the lights and signs rather than by searching the lot.

The architectural mapping. The components of the mall parking system map directly onto the components of Constitutional Storage:

- The per-space indicator lights are the *Spaces table*. They encode free-space state per extent, independently of any object identity. A green light asserts that the extent is free; a red light asserts that it is occupied. The lights are derived from physical occupancy state, not generated independently.
- The per-floor aggregate signs are *Spaces-table aggregates over jurisdictions*. Each floor is a jurisdiction; the sign reports the count of free extents within that jurisdiction. Drivers can make routing decisions at the floor granularity without consulting individual space lights.
- The space numbers are the *Things table*. They allow a driver who has parked a specific car (an object with identity) to locate it again by identifier rather than by spatial search. The Things table answers "where is my car?"; the Spaces table answers "where is any free spot?".
- The act of parking is a *transition* (free -> occupied). The detection layer admits the transition by updating the per-space state, which propagates to the indicator light. The act of leaving is the inverse transition (occupied -> free). Both transitions update the Spaces-table state without modifying the Things-table identifier the space carries.
- The driver's decision process is *consultation of the constitutional layer before action*. A driver reads the per-floor sign to choose a jurisdiction, then reads per-space lights within that jurisdiction to choose an extent, then performs the parking transition. At no point does the driver probe densely through occupied spaces looking for one that happens to be free.

What the parking solution gets right. The conventional dense-storage failure mode this paper opens with — circling endlessly through a near-full table looking for a slot that may not exist, with no global free-space information available — is precisely the failure mode that mall parking infrastructure eliminates. A driver in a smart parking system does not greedy-probe; the driver consults the Spaces-table state directly and routes to a jurisdiction where free space is abundant. *Empty space is exposed as legible state, not encountered through dense search.* This is the architectural commitment the present paper makes for hash tables, allocators, file systems, and object stores.

The per-floor aggregate signs further demonstrate that the pattern composes naturally across jurisdictional granularity. A driver who does not care which

specific space they get but cares about avoiding crowded floors uses the level-aggregate sign as a routing primitive. "The top floor has hundreds of free spaces; the lower floors have a few; I will route to abundance and accept the walk." This is the routing logic of a federated allocator that prefers high-availability jurisdictions, expressed at human scale.

The integration gap. The architectural pattern of exposing free-space state as legible constitutional information has been deployed in physical parking infrastructure for decades. Systems-theory work on hash tables, allocators, and storage has extensively developed individual mechanisms — probing strategies, allocation classes, compaction algorithms, garbage collection, tiering policies, replication management, write-ahead logging, snapshot regimes, lifecycle policies — but has rarely reconstructed these mechanisms as components of a *single constitutional specification* in which occupancy, absence, movement, repair, authority, and receipts are governed together. The present paper does not claim to discover individual storage mechanisms; it claims to offer their reconstruction as a unified constitutional specification.

Caveats on the analogy. The mall parking system is a partial instantiation of Constitutional Storage, not a complete one. It implements the Things-table / Spaces-table separation and the constitutional indexing layer, but it does not implement receipts (parking events are not durably auditable in most deployments), federation (one mall's parking system does not interoperate with another's), authority artifacts (there are no capabilities or delegations beyond the physical access controls), or governed amendments to the constitution itself (the lighting policy is fixed by the mall, not subject to constitutional revision).

A further limit of the analogy is on the Things-table side. Cars in a parking system cannot be moved to colder jurisdictions, compressed for denser storage, deduplicated against other cars, or replicated across multiple spaces. Digital artifacts can. The parking analogy strongly demonstrates the lookup/allocation separation and the Spaces-table indexing layer, but it cannot demonstrate the *occupancy-mobility* dimension that distinguishes digital storage from physical parking. The Things table in digital storage is substantially more constitutionally active than the spot-numbering scheme in parking infrastructure suggests: digital objects are subject to retention policies, tiering obligations, deduplication rules, movement authorities, and lifecycle constraints that have no parking-lot analogue. The reasons the analogy cannot fully demonstrate the model are the same reasons digital storage especially benefits from it: mobility, deduplication, tiering, and versioning are precisely the constitutional surfaces that integrated governance is designed to discipline, and digital systems have far more such surface than parking systems do. The occupied-space governance developed in Section 7.4 is the part of the architecture the analogy cannot show.

The analogy is useful for demonstrating the core pattern; the formal architecture extends the pattern with the additional machinery that digital storage systems can support.

---

3. The Failure of Collapsed Storage

A collapsed storage system asks one structure to answer too many questions.

In a simple open-addressing hash table, the array is expected to answer:

```
What keys exist?
Where are they?
Where is there room for new keys?
How can a future lookup find a key?
How much fullness can the table tolerate?
What repair, if any, is required after insertion?
```

In a simple file allocator, the storage layer may be expected to answer:

```
What files exist?
Where are their bytes?
Which regions are free?
Which holes are contiguous?
Can an update preserve locality?
Can deletion avoid fragmentation?
When should compaction occur?
```

In a governed artifact system, still more questions arise:

```
Who authorized this mutation?
What semantic object does this byte sequence represent?
Is the object current, deleted, superseded, or historical?
Can the state be replayed?
Can the artifact move without changing identity?
Can trivial operations bypass heavy receipt overhead?
```

The collapse is convenient because it hides the questions.

It is dangerous because hidden questions do not disappear. They return as performance cliffs, stale metadata, fragmented heaps, ambiguous deletion, recovery failure, and audit gaps.

Constitutional Storage rejects this collapse.

It does not ask one structure to be the filing cabinet, parking map, legal record, repair schedule, and traffic policy.

It separates those roles.

---

4. Constitutional Storage Model

A Constitutional Storage system is defined as:

```
CS = (A, T, S, Gamma, R, Omega, Lambda)
```

where:

- `A` is the artifact arena, the physical or logical space in which artifact bodies may reside.
- `T` is the Things table, mapping artifact keys to descriptors.
- `S` is the Spaces table, representing free extents and storage-health summaries.
- `Gamma` is the storage constitution, deciding admissibility of mutations and repairs.
- `R` is the receipt ledger, recording admitted transitions and repair obligations.
- `Omega` is the ordering relation over storage events.
- `Lambda` is the lane policy, distinguishing full governance, batched governance, repair governance, and fast-lane governance.

A storage transition has the form:

```
(A_t, T_t, S_t, p)  =>Gamma  (A_{t+1}, T_{t+1}, S_{t+1}, r, pi)
```

where:

- `p` is a storage petition,
- `r` is a receipt,
- `pi` is the proof or explanation that the transition was admitted.

The constitution does not merely ask:

```
Can these bytes be written?
```

It asks:

```
Is this mutation admissible under the current state?
Does it preserve descriptor correctness?
Does it preserve free-space correctness?
Does it create unacceptable fragmentation?
Does it require immediate repair?
May repair be deferred?
Which lane may process it?
What receipt must be emitted?
```

Thus storage becomes authorized transformation rather than raw mutation.

---

5. Things and Spaces

The central architectural separation is between **Things** and **Spaces**.

The Things table records what exists:

```
ThingDescriptor {
    key
```

```
    address
    length
    state_hash
    content_hash?
    meaning_hash?
    admissibility_hash?
    locality_hash?
    version
    receipt_ref
    lane
    flags
}
```

The Spaces table records what remains available:

```
FreeExtent {
    start
    length
    class?
    region?
    mobility_constraints?
    repair_status?
}
```

A lookup asks the Things table:

```
lookup(key) -> address, length
```

An allocation asks the Spaces table:

```
allocate(size, constraints) -> start, length
```

This separates two questions that open addressing deliberately binds together:

```
Where can the object be placed?
How will the object be found later?
```

In Constitutional Storage, the first is an allocation question.

The second is an indexing question.

They may be optimized differently, governed differently, compacted differently, and repaired differently.

5.1 Minimal Example

Initial state:

```
Arena A:
    [0..1023] free

Things T:
```

```
    {}

Spaces S:
    FreeExtent(start=0, length=1024)
```

Create artifact `X` of length `128`:

```
petition:
    Create(X, length=128)

post-state:
    Things:
        X -> ThingDescriptor(address=0, length=128, state_hash=H(X))

    Spaces:
        FreeExtent(start=128, length=896)

receipt:
    R_create_X
```

Create artifact `Y` of length `64`:

```
Things:
    X -> address=0, length=128
    Y -> address=128, length=64

Spaces:
    FreeExtent(start=192, length=832)
```

Delete `X`:

```
Things:
    Y -> address=128, length=64

Spaces:
    FreeExtent(start=0, length=128)
    FreeExtent(start=192, length=832)

receipt:
    R_delete_X
```

Repair moves `Y` to reduce fragmentation:

```
move:
    Y from [128..191] to [0..63]

Things:
    Y -> address=0, length=64

Spaces:
```

```
    FreeExtent(start=64, length=960)

receipt:
    R_move_Y
```

The content of `Y` did not change.

Its placement changed.

The receipt records which axis changed.

This is the whole architecture in miniature: things, spaces, movement, deletion, and repair become separately legible.

---

6. Storage Invariants

A Constitutional Storage system must preserve a compact set of invariants.

6.1 Coverage

Every addressable unit in the arena must be classified:

```
unit in occupied or unit in free or unit in reserved
```

No unit may be unaccounted for.

6.2 Disjointness

Free extents and occupied extents must not overlap.

```
forall f in S, forall d in T:
    extent(f) intersect extent(d) = empty
```

6.3 Ownership

Every occupied extent must be owned by exactly one current descriptor unless explicitly governed as shared, deduplicated, or copy-on-write.

```
occupied_extent -> unique owner
```

6.4 Descriptor Correctness

Every descriptor must point to bytes whose state commitment matches the descriptor.

```
H(storage[address:length]) = descriptor.state_hash
```

6.5 Movement Preservation

If an artifact is moved without content or semantic change, its content or state commitment must remain stable, and the movement must be receipted as placement change rather than content change.

6.6 Coalescence

Adjacent free extents must either be coalesced or explicitly recorded as non-coalescible for a constitutional reason.

```
Free[a, n] + Free[a+n, m] -> Free[a, n+m]
```

6.7 Atomicity

Mutations touching Things, Spaces, receipts, and the arena must commit atomically or recover deterministically.

6.8 Repair Boundedness

Temporary disorder is admissible only if bounded by explicit thresholds and accompanied by repair obligations.

6.9 Lane Containment

Fast-lane operations must remain within their authorized scope or escalate.

A fast lane that silently handles non-trivial state is not a fast lane.

It is a breach.

6.10 Capacity with Jurisdiction

Free space is not undifferentiated. Modern storage systems already maintain differentiated capacity informally: an emergency reserve held back from ordinary allocation, a repair-debt reserve held against compaction obligations, a failure-domain reserve held against replication recovery, a burst reserve held for traffic spikes, a legal-retention reserve held against regulatory obligations, a migration reserve held against pending tiering moves, a fast-lane budget held for trivial operations. These reserves are typically operational rather than constitutional: they exist as configuration thresholds, allocation watermarks, or scheduling priorities, but they are not always specified as jurisdictionally distinct capacity classes whose admissibility rules differ.

Constitutional Storage treats capacity as *jurisdictional*. Each constitution decides which capacity jurisdictions to recognize, with what admissibility rules, with what reserve discipline, with what crossover authority, and with what reconciliation obligations. *The framework does not prescribe a canonical reserve typology*; it

requires that whatever reserves a constitution chooses to maintain be specified as jurisdictional capacity classes with explicit constitutional rules, not as informal operational thresholds.

This composes naturally with the lane containment invariant of Section 6.9: fast-lane budget is one jurisdiction of capacity among several, and its containment rules are one instance of the broader pattern that capacity jurisdictions have explicit boundary policies.

6.11 Mechanical Durability versus Constitutional Legitimacy

A state recovered from write-ahead logs, snapshots, replicas, or erasure-coded reconstruction may be *byte-identical* to a valid prior state and yet *constitutionally illegitimate*. The mechanical durability of storage — the property that bytes survive failure and can be reconstructed — is necessary but not sufficient for constitutional admissibility. A recovered state may exhibit invalid allocations (Things-table entries whose Spaces-table extents are no longer consistent), unbounded repair debt (compaction obligations exceeding any constitutional deadline), unlicensed fast-path leakage (lane budgets exceeded without reconciliation), expired retention (objects past their constitutional lifetime), missing receipts (occupancy whose original admission is no longer producible), or ambiguous authority (delegations whose chain of attestation has been broken in the recovery path).

Constitutional Storage distinguishes these two properties explicitly. *Mechanical durability* is the substrate-level property that storage state can be reconstructed after failure. *Constitutional legitimacy* is the framework-level property that the reconstructed state satisfies the framework's admissibility predicates: invariant compliance, receipt producibility, authority chain integrity, repair debt boundedness, lane containment, and capacity-jurisdiction discipline. Recovery must produce both. A recovery process that produces byte-identical state but breaks constitutional legitimacy has succeeded mechanically and failed constitutionally; the framework treats this as a recovery failure requiring constitutional remediation, not as a successful recovery requiring no further action. Constitutional remediation is itself a governed response: it takes the form of receipted repair, receipted rollback to a previously legitimate state, or receipted replay against the constitution in force at the time of the original transitions, with the remediation transition's own receipt recording what was restored, against which constitution, and by what authority. Mechanical recovery without governed remediation leaves the system byte-faithful but constitutionally adrift; mechanical recovery followed by governed remediation restores both substrate and standing.

This distinction matters in three operational contexts. *Forensics*: explaining what a system was permitted to do at a given time requires constitutional legitimacy, not just mechanical reconstruction. *Compliance*: regulated workloads require that recovered state be auditable under the constitution that governed its creation, not merely that the bytes have been restored. *Long-term maintainability*: byte-

faithful archives whose constitutional context has been lost are mechanically durable but operationally fragile, because future actions taken against them lack the legitimacy frame against which they could be evaluated.

The framework's commitment is that recovery operations must restore *both* mechanical state and constitutional legitimacy, and that the receipt of a recovery operation is itself a governed transition whose admissibility is checked against the constitution in force at the time of recovery.

---

7. Three Costs, Three Governance Mechanisms

Constitutional Storage separates three different costs that are often blurred.

7.1 Physical Disorder: Repair Governance

Physical disorder concerns the shape of storage itself: fragmented free space, scattered extents, pinned regions, and failed contiguity. It is governed through metrics, repair obligations, compaction policies, and movement receipts.

Repair governance answers:

```
What shape is the storage in?
What shape must it remain in?
What disorder may persist temporarily?
What repair is owed?
```

7.2 Receipt Overhead: Batch Governance

Receipt overhead concerns the cost of proving many small operations. It is governed through batches, Merkle roots, aggregate receipts, and inclusion proofs.

Batch governance answers:

```
Which operations may be grouped?
What aggregate state transition did they produce?
Can each operation still be proven inside the batch?
```

7.3 Semantic Significance: Fast-Lane Governance

Semantic significance concerns whether an operation is trivial enough to use compact machinery without full procedure. It is governed through lane scope, lane authority, budgets, reconciliation, and escalation.

Fast-lane governance answers:

```
Is this operation truly trivial?
Who authorized reduced procedure?
How much unreconciled state may accumulate?
When must the operation escalate?
```

These mechanisms may interact, but they do not answer the same question. Repair governs physical layout. Batching governs proof cost. Fast lanes govern procedural weight.

7.4 Occupancy Legitimacy: Governance of Occupied Space

The three governance mechanisms above — repair, batch, fast-lane — address how storage state is *changed* under constitutional discipline. They do not yet address why specific state is *legitimate in the first place*. Free-space failure is often downstream of occupied-space failure. A system runs out of usable space not only because holes are poorly tracked, but because occupied regions have unclear legitimacy: stale objects persist past their justification, cold objects remain in hot storage, pinned objects are not constitutionally justified for the pinning, replicas exceed policy, legal holds are ambiguous, deletion is incomplete, and objects that should be moved, compressed, deduplicated, or demoted to colder jurisdictions remain sovereign in their current placement.

The emergency is not that the system cannot find a hole. The emergency is that it never governed why the full spaces deserved to remain full.

Occupied-space governance as a constitutional question. Occupancy legitimacy is the Things-table analogue of what Spaces-table governance has done for free-space. Where Spaces governance asks *where is space free, under what jurisdiction, and with what reserve discipline?*, occupancy governance asks *why does each occupied extent currently retain its space, under what authority, with what expiration, and under what movement obligation?* Both are constitutional questions about state legitimacy; both produce admissibility decisions about transitions; both are settled through receipts and reconciled through repair governance. The framework treats them symmetrically.

Failure modes of unjustified occupancy. Concrete failures attributable to ungoverned occupancy include:

- *Stale retention*: objects retained past their constitutionally justified lifetime, often because expiration policy was never declared or because deletion semantics were ambiguous.
- *Tier inversion*: cold objects occupying hot storage, or hot objects exiled to cold storage, because movement authority was never granted and tiering policy was never enforced.
- *Unjustified pinning*: objects marked immovable without a constitutional reason for the pinning, often because pinning was an operational shortcut whose justification decayed but whose status persisted.
- *Replica excess*: more copies of an object than policy permits, accumulated because replication mechanisms ran without retention reconciliation.
- *Ambiguous holds*: legal, regulatory, or compliance holds applied to objects without clear scope, expiration, or release authority, leading to permanent

quasi-retention.

- *Incomplete deletion*: objects whose primary descriptors are removed but whose replicas, snapshots, derived indexes, or auxiliary structures persist, leaving the system in a state where the object is partially gone.
- *Under-receipted authority*: occupancy whose original constitutional basis is no longer producible — the receipt of admission was lost, never created, or held only by a party with no incentive to retain it.

Each of these failures is a constitutional question about the Things table that the conventional storage stack often handles operationally, partially, or not at all. Constitutional Storage requires that each be governed explicitly: every occupied extent should have a constitutionally derivable answer to *why this object, in this place, under what authority, until when?*

Fast lanes as licensed jurisdictions, not lawless shortcuts. Section 7.3 introduced fast-lane governance as a mechanism for trivial operations to use compact primitives while remaining under constitutional scope. The occupancy-legitimacy framing sharpens this characterization. A governed fast lane is a *licensed jurisdiction* with explicit scope, debt budget, reconciliation deadlines, and escalation rules, rather than a *lawless shortcut* whose disorder eventually leaks into the main system. The constitutional distinction is exactly the distinction between governed and ungoverned occupancy: a fast-lane object occupies its space under explicit, time-bounded, reconciliation-obligated authority, not under the silent assumption that the shortcut will eventually be cleaned up.

The architectural symmetry. The Three Costs (physical disorder, receipt overhead, semantic significance) and their corresponding governance mechanisms apply to occupancy legitimacy as much as to absence management. Stale retention produces physical disorder (unrecoverable space) and should be subject to repair governance. Per-object legitimacy verification incurs receipt overhead and should be subject to batch governance. Trivial occupancy events (cache writes, ephemeral artifacts, in-flight buffers) carry minimal semantic significance and should be eligible for fast-lane governance with reconciliation obligations. The three governance mechanisms generalize from absence-management to a unified treatment of occupancy and absence as parallel constitutional states.

---

8. Repair Governance

Fragmentation is not merely scattered free space. It is a storage state in which future admissible transitions become more expensive or impossible.

A fragmented system may have enough total free space and still be unable to admit an allocation requiring contiguity:

```
total_free = 1024
largest_free_extent = 12
request = 128
```

Naively, this is failure.

Constitutionally, it is debt.

Define:

```
fragmentation_score(S)
free_extent_count(S)
largest_free_extent(S)
unrepaired_bytes(S)
repair_deadline(S)
pinned_fragmentation_score(S)
```

A constitution may set thresholds:

```
fragmentation_score(S) <= F_max
free_extent_count(S) <= E_max
largest_free_extent(S) >= L_min
unrepaired_bytes(S) <= B_max
repair_deadline(S) <= T_max
```

A mutation is admissible if it either preserves these bounds or emits a bounded repair obligation:

```
admit(p) iff
    preserves_health(p, S)
    or creates_bounded_repair_obligation(p, S)
```

A repair operation may perform:

```
move artifact
split artifact
merge extents
rewrite descriptors
compress pages
evacuate arena
rebalance lanes
reclassify hot/cold objects
```

A movement receipt has the form:

```
MoveReceipt {
    key
    old_address
    new_address
    length
    content_hash_before
    content_hash_after
    reason
    policy
    prior_receipt
```

```
}
```

The required invariant is:

```
content_hash_before = content_hash_after
```

unless the operation is explicitly a content-changing update.

Thus physical movement does not silently become semantic mutation.

---

9. Batch Governance

Batching addresses proof and receipt overhead, not physical fragmentation.

A batch is a governed set of operations:

```
Batch {
    operations[]
    scope
    aggregate_prestate
    aggregate_poststate
    merkle_root
    policy
}
```

Instead of issuing one heavy receipt per tiny operation, the system may issue one aggregate receipt with inclusion proofs:

```
BatchReceipt {
    batch_root
    operation_count
    pre_state_root
    post_state_root
    policy
    proof_summary
}
```

Each operation remains individually provable through Merkle inclusion, while the overhead is amortized across the batch.

Batch governance is appropriate for:

```
tiny writes
counters
temporary cache entries
hot index updates
short-lived derived state
fast-lane reconciliation
```

Batching does not remove governance.

It changes governance granularity.

---

10. Fast-Lane Governance

Ultra-compact structures remain valuable.

Dense hash tables, compact arrays, slab allocators, bitmaps, caches, Bloom filters, and hot-path indexes may outperform full constitutional descriptors for trivial operations.

Constitutional Storage therefore introduces a governed fast lane.

A fast lane is not outside the constitution. It is a constitutionally authorized jurisdiction with reduced procedure.

```
FastLane {
    scope
    authority
    jurisdiction
    operation_class
    representation
    budget
    reconciliation_rule
    escalation_rule
    audit_rule
}
```

An operation may enter the fast lane only if it satisfies lane policy:

```
fast_admit(op, S, Lambda) iff
    op.class in Lambda.allowed_classes
    and op.size <= Lambda.max_size
    and op.risk <= Lambda.max_risk
    and op.semantic_weight <= Lambda.max_semantic_weight
    and op.scope subseteq Lambda.scope
    and lane_debt <= Lambda.max_debt
```

If any condition fails, the operation escalates:

```
fast lane -> batched lane -> full lane -> administrative review
```

Fast-lane state must periodically reconcile with canonical state. Reconciliation checks:

```
fast-lane state root
operation summary
budget consumption
fragmentation contribution
descriptor consistency
```

```
scope compliance
semantic-weight compliance
```

The result may be:

```
confirm lane state
promote objects to full lane
compact lane
flush lane
quarantine lane
revoke lane authority
emit violation receipt
```

The fast lane is lawful because its boundaries are known.

It may move quickly because it knows where it must stop.

---

11. Mutation Semantics

CRUD operations are not primitive effects. They are petitions whose meaning depends on which constitutional axis changes.

11.1 Create

A create petition requests admission to allocate space, write bytes, construct a descriptor, update free-space state, and emit a receipt.

11.2 Read

A read petition may be trivial, privileged, audited, private, or semantic depending on the artifact and jurisdiction. Reads may enter a fast lane when they do not alter state and do not disclose protected material.

11.3 Update

An update petition may be in-place, append-only, copy-on-write, page-local, relocation-based, or semantically transformative.

The constitution distinguishes:

```
same identity, changed content
same content, changed placement
same meaning, changed representation
superseded artifact
partial deletion
versioned mutation
```

11.4 Delete

Deletion is not disappearance.

A delete petition changes current state while preserving historical legibility. The constitution decides whether deletion means:

```
remove current descriptor
tombstone descriptor
preserve receipt only
preserve content hash
preserve semantic identity
permit address reuse
forbid address reuse
create new version
```

A deleted artifact may cease to occupy current state while remaining valid historical evidence.

---

12. Partial Deletion and Normalization

Partial deletion is where naive storage often becomes constitutionally ambiguous.

Suppose a file occupies:

```
File A = [0..99]
```

A petition deletes the first half:

```
delete [0..49]
```

A naive system might leave:

```
free [0..49]
File A remainder [50..99]
```

Constitutional Storage instead asks what the operation means.

If the content itself has changed, then the result is a new current version:

```
File A v1 = [0..99]
File A v2 = [50..99]
```

If placement is normalized after deletion, the remaining content may be moved:

```
move [50..99] -> [0..49]
File A v2 -> address 0, length 50
free [50..99]
```

This is not merely deletion. It is:

```
content transition
placement transition
```

```
free-space transition
receipt transition
```

The question is not simply:

```
Which bytes remain?
```

The constitutional question is:

```
What object now exists, where does it live, why is that transition
admissible, and what history remains?
```

---

13. Recursive Index Governance and Small Artifacts

The Things table itself must be stored somewhere.

This creates a recursion:

```
artifact bodies need descriptors
descriptors need indexes
indexes need storage
storage needs descriptors
```

Constitutional Storage handles this by classifying indexes as governed artifacts with special roles. Possible implementations include:

```
sparse hash table
dense open-addressed hash table
B-tree
LSM tree
trie
perfect hash table
bitmap-indexed slab
append-only log with checkpointed index
```

The constitution need not choose one representation globally. It may assign representations by jurisdiction:

```
tiny hot keys:
    dense fast-lane hash table

large durable artifacts:
    full descriptor index

range queries:
    B-tree

write-heavy streams:
    LSM
```

```
immutable archives:
    perfect hash or content-addressed map
```

Small artifacts are the hardest case for full constitutional descriptors. If an artifact is 16 bytes and its descriptor is 64 bytes, overhead dominates.

The model responds through granularity adaptation:

```
inline values
slab pages
micro-batches
page-level receipts
Merkleized subrecords
fast-lane indexes
semantic grouping
temporary lanes
```

Tiny artifacts may become subrecords inside a governed page:

```
PageDescriptor {
    page_id
    address
    length
    subrecord_root
    policy
}
```

The open-addressing problem therefore remains important as a local primitive.

It is not abolished.

It is licensed.

---

14. Concurrency and Recovery

Concurrent writers can double-allocate the same extent unless allocation is governed.

The constitution must prevent:

```
Writer A consumes extent X
Writer B consumes extent X
```

Possible mechanisms include:

```
single-writer log
extent locks
compare-and-swap claims
optimistic transactions
```

```
epochs
lease-based allocation
sharded arenas
conflict receipts
```

A concurrent allocation is admissible only if its claimed extent remains free at commit time.

```
admit(allocation) iff
    extent in free_state_at_commit
```

Crash recovery is the same problem under interrupted time.

A storage transition may touch:

```
artifact body
Things table
Spaces table
receipt ledger
repair obligations
lane budgets
```

A crash between these updates can create contradiction. Constitutional Storage therefore requires transactional structure:

```
prepare
reserve extent
write body
update descriptor
update spaces
emit commit receipt
publish state root
```

The governing invariant is:

```
No current state without a receipt.
No receipt without reconstructible state.
```

---

15. Threat Model

Constitutional Storage distinguishes four major threat classes.

15.1 Benign Fragmentation

Benign fragmentation arises from ordinary mixed-size create, update, and delete operations. It is handled through coalescence, repair obligations, batching, and background compaction.

15.2 Adversarial Fragmentation

Adversarial fragmentation arises when a workload intentionally creates pathological layouts:

```
allocate large
allocate small
allocate large
allocate small
delete large
```

The constitution may respond with allocation-class separation, slab isolation, mobility requirements, repair surcharges, lane revocation, or workload quarantine.

15.3 Pinned-Object Fragmentation

Pinned objects cannot always move. Objects may be pinned because of external references, memory mapping, hardware constraints, real-time access, legal hold, active transactions, or cross-jurisdictional anchoring.

Pinned objects require mobility classes:

```
movable
temporarily pinned
permanently pinned
movable only through indirection
movable only with authority
```

15.4 Unlicensed Fast-Path Behavior

A fast lane becomes dangerous when it handles operations outside its scope. The threat is not speed. The threat is hidden significance.

Unlicensed fast-path behavior includes:

```
semantic objects treated as trivial
authority-bearing state cached without reconciliation
fast-lane debt exceeding budget
cross-jurisdictional operations handled locally
compact indexes becoming canonical without receipts
```

The remedy is escalation, quarantine, reconciliation, and possible revocation of lane authority.

---

16. Physical Cost

Governance cannot abolish physics.

Compaction moves bytes. Receipts consume space. Indexes consume memory. Batches need reconciliation. Fast lanes need audits.

The constitution can govern these costs but not erase them.

Storage policy should therefore include budgets:

```
max_write_amplification
max_compaction_bytes_per_operation
max_background_repair_rate
max_receipt_growth
max_latency
max_media_wear
```

A repair may be admissible in one regime and inadmissible in another.

For example:

```
SSD with limited endurance:
    defer compaction aggressively

archival magnetic storage:
    compact in large scheduled windows

memory arena:
    compact freely if references are relocatable

real-time system:
    avoid latency spikes; use incremental repair
```

The constitutional question is not:

```
Can disorder be removed for free?
```

It is:

```
When is the cost of repair admissible?
```

---

17. Relationship to Open Addressing

Yao's analysis and the later Krapivin-Farach-Colton-Kuszmaul result concern a deliberately constrained model: open addressing, probing, load, and the cost of finding items in dense tables.

This model is artificial in the productive sense. It isolates the physics of one low-level province.

Constitutional Storage does not claim that this province is meaningless.

It claims that the province should not be mistaken for the whole kingdom.

Open addressing asks:

```
How fast can lookup remain when the array itself is the main
representation?
```

Constitutional Storage asks:

```
Which representation should govern lookup, placement, absence,
repair, and authority?
```

These are different questions.

A mature system may use elastic hashing or related dense-table techniques inside the Things table, inside fast lanes, inside caches, or inside compact metadata pages. But it should not force every artifact body to live under the same constraint.

The result is architectural peace:

```
open addressing:
    primitive

constitutional storage:
    system

fast lane:
    licensed primitive inside the system
```

Elastic Hashing teaches that greed can be locally rational and globally destructive.

Taking the first open slot seems efficient.

But taking every first open slot can erase the structure that future operations need.

The generalized storage rule is:

```
Do not admit every mutation merely because it is mechanically possible.
```

The firebreak is a constitutional intuition in algorithmic form.

---

18. Propositions

Proposition 1: Absence Is State

A storage architecture that cannot explain its empty spaces cannot fully explain its occupied spaces.

The constitution of things requires a constitution of holes.

Proposition 2: Separation of Things and Spaces Dissolves Blind Allocation

If object lookup is performed through descriptors and allocation is performed through free-space extents, then artifact bodies need not be discoverable through their own placement geometry.

This dissolves the full-parking-lot problem for artifact bodies.

It does not dissolve the need for efficient descriptor indexes.

Proposition 3: Fragmentation Becomes Governable When Measured

If a storage constitution defines fragmentation metrics, thresholds, and repair obligations, then fragmentation can be admitted, denied, deferred, or repaired as a policy-governed state rather than an unclassified side effect.

Proposition 4: Batch Governance Amortizes Legibility

If small operations are aggregated under a verifiable batch receipt, governance overhead can be amortized without making individual operations unverifiable.

Proposition 5: Fast Lanes Preserve Efficiency Only When Scoped

A low-overhead lane remains constitutional only if its authority, jurisdiction, operation class, debt budget, reconciliation rule, and escalation rule are explicit.

A fast lane without reconciliation is not a lane.

It is an escape.

Proposition 6: Dense Hashing Remains a Local Primitive

Ultra-compact indexes remain useful inside fast lanes and descriptor layers. Constitutional Storage subsumes them architecturally but does not render their theory irrelevant.

Proposition 7: Governance Does Not Remove Physical Costs

Compaction, receipts, indexes, and audits consume resources. Constitutional Storage does not eliminate these costs. It makes them admissible, bounded, scheduled, and attributable.

---

19. Limitations

Constitutional Storage can become its own overhead if lane boundaries are too broad, receipts are too frequent, repair obligations are too strict, or trivial operations are misclassified as meaningful.

The model therefore depends on disciplined granularity.

Compact primitives remain necessary. Dense hash tables, bitmaps, slabs, caches, and specialized indexes should remain available where the constitution grants them authority. The purpose of Constitutional Storage is not to replace those primitives, but to prevent them from silently becoming the whole governance model.

The architecture also depends on honest metrics. A poor fragmentation score, weak semantic-weight classifier, or vague fast-lane policy can create false legitimacy. Constitutional representation clarifies the questions; it does not answer them automatically.

The constitution itself must be designed, audited, revised, and constrained.

Otherwise the map becomes another maze.

---

20. Conclusion

The full parking lot is not merely full.

It is underdescribed.

It lacks a constitutional account of why some spaces remain occupied, which absences remain available, which objects may move, which reserves may be consumed, and which transitions are legitimate.

Constitutional Storage begins by refusing to let one structure answer all questions. It separates things from spaces, placement from identity, mutation from admissibility, and history from current authority. It treats fragmentation as debt, defragmentation as repayment, batching as amortized legitimacy, and fast lanes as authorized low-friction jurisdictions rather than lawless shortcuts.

This does not make storage free.

It makes storage accountable.

Small artifacts still require careful granularity. Indexes still require efficient implementation. Compaction still costs writes. Concurrency still needs ordering. Crashes still require recovery. But these are no longer hidden ghosts in the machine. They are named constitutional conditions with policies, thresholds, receipts, and escalation paths.

A trivial operation may move quickly.

A meaningful operation must justify itself.

A broken space must repair itself or confess its debt.

The firebreak taught us that an empty slot can save the future.

Constitutional Storage replies:

so can a justified occupancy, a governed reserve, and a receipted transition.

Appendix A: Minimal Data Model

```
ThingDescriptor {
    key: Key
    address: Address
    length: Length
    state_hash: Hash
    content_hash: Optional<Hash>
    meaning_hash: Optional<Hash>
    admissibility_hash: Optional<Hash>
    locality_hash: Optional<Hash>
    version: Version
    lane: LaneId
    receipt_ref: ReceiptRef
    flags: Flags
}

FreeExtent {
    start: Address
    length: Length
    region: RegionId
    class: SizeClass
    mobility: MobilityClass
    repair_status: RepairStatus
}

RepairObligation {
    id: ObligationId
    kind: RepairKind
    affected_range: ExtentSet
    created_by: ReceiptRef
    deadline: LogicalTime
    bound: RepairBound
    priority: Priority
}

LanePolicy {
    lane_id: LaneId
    authority: Authority
    jurisdiction: Scope
    allowed_classes: OperationClass[]
    max_size: Size
    max_risk: Risk
    max_semantic_weight: SemanticWeight
```

```
    max_debt: Debt
    reconciliation_interval: Interval
    escalation_rule: Rule
}
```

Appendix B: Storage Health Metrics

A constitution may define:

```
total_free(S)
largest_free_extent(S)
free_extent_count(S)
mean_free_extent_size(S)
fragmentation_score(S)
pinned_fragmentation_score(S)
descriptor_overhead_ratio(S)
write_amplification(S)
receipt_overhead_ratio(S)
lane_debt(S)
unrepaired_bytes(S)
```

Example:

```
fragmentation_score(S) =
    1 - (largest_free_extent(S) / total_free(S))
```

This simple formula is not universal, but it captures the intuition that free space is healthier when concentrated rather than shattered.

A more complete metric may include pinned objects:

```
effective_fragmentation_score(S) =
    1 - (largest_movable_repairable_extent(S) / total_free(S))
```

This distinguishes ordinary fragmentation from fragmentation caused by immovable anchors.

Appendix C: Admission Sketches

C.1 Create Admission

```
function admit_create(key, bytes, state):
    if !authorized(key, create):
        deny

    lane = choose_lane(key, bytes, state)
```

```
    if lane == FAST:
        if fast_admit(create, state):
            admit_fast
        else:
            escalate

    extent = find_extent(len(bytes), state.Spaces)

    if extent == none:
        repair_plan = plan_compaction(len(bytes), state)
        if repair_plan.admissible:
            admit_with_repair(repair_plan)
        else:
            deny

    if fragmentation_after(create) > F_max:
        obligation = create_repair_obligation()
        if obligation.bounded:
            admit_with_obligation
        else:
            deny

    admit_full
```

C.2 Defragmentation Admission

```
function admit_defrag(plan, state):
    require authorized(plan.actor, defrag)

    require all moved objects are movable

    require no extent overlap in plan.post_state

    require descriptor_updates match moved objects

    require content_hash_before == content_hash_after
        for all placement-only moves

    require post_fragmentation_score <= pre_fragmentation_score
        or plan.justification explains exception

    require crash_recovery_path exists

    emit MoveReceipts
    emit DefragReceipt
```

```
    admit
```

C.3 Fast-Lane Reconciliation

```
function reconcile_fast_lane(lane, state):
    root = compute_lane_state_root(lane)

    verify lane operations within scope

    verify debt <= max_debt

    verify no promoted object lacks descriptor

    verify no semantic object remained hidden in fast lane

    if all checks pass:
        emit ReconciliationReceipt

    else:
        quarantine lane
        revoke or suspend lane authority
        escalate unresolved operations
        emit ViolationReceipt
```

Appendix D: Problem-to-Solution Matrix

- **Problem:** Full parking lot for artifact bodies
  Constitutional Solution: Separate Things lookup from Spaces allocation
  Remaining Cost: Descriptor index still needed
- **Problem:** Free-space discovery
  Constitutional Solution: Spaces table with extent lengths
  Remaining Cost: Spaces table maintenance
- **Problem:** Free-space table bloat
  Constitutional Solution: Coalescing, compaction, summaries, page maps
  Remaining Cost: Repair overhead
- **Problem:** Fragmentation
  Constitutional Solution: Fragmentation metrics and repair obligations
  Remaining Cost: Write amplification
- **Problem:** Partial deletion
  Constitutional Solution: Versioned mutation plus normalization
  Remaining Cost: Descriptor churn

- **Problem:** Moving artifacts
  Constitutional Solution: Movement receipts and descriptor updates
  Remaining Cost: Atomicity requirement
- **Problem:** Small-artifact overhead
  Constitutional Solution: Batching, slabs, inline values, fast lanes
  Remaining Cost: Less per-object audit detail
- **Problem:** Receipt bloat
  Constitutional Solution: Batch receipts and Merkle inclusion
  Remaining Cost: Proof complexity
- **Problem:** Concurrency
  Constitutional Solution: Ordering, locks, epochs, transactional claims
  Remaining Cost: Coordination overhead
- **Problem:** Crash inconsistency
  Constitutional Solution: Prepare/commit/replay discipline
  Remaining Cost: Log and recovery cost
- **Problem:** Physical media wear
  Constitutional Solution: Repair budgets and scheduled compaction
  Remaining Cost: Slower repair
- **Problem:** Ultra-compact use case
  Constitutional Solution: Governed fast lane
  Remaining Cost: Reconciliation burden
- **Problem:** Adversarial fragmentation
  Constitutional Solution: Allocation-class policy and workload quarantine
  Remaining Cost: Policy complexity
- **Problem:** Pinned objects
  Constitutional Solution: Mobility classes and repair routing
  Remaining Cost: Reduced compaction freedom
- **Problem:** Greedy local mutation
  Constitutional Solution: Health-aware admission and firebreak preservation
  Remaining Cost: Policy computation
- **Problem:** Hidden triviality
  Constitutional Solution: Fast-lane classification and escalation
  Remaining Cost: Classification burden
- **Problem:** Unjustified occupancy
  Constitutional Solution: Occupancy legitimacy checks and lifecycle authority
  Remaining Cost: Policy complexity

- **Problem:** Emergency reserve misuse
  Constitutional Solution: Capacity jurisdictions and reserve admission rules
  Remaining Cost: Reduced ordinary capacity
- **Problem:** Mechanical recovery without legitimacy
  Constitutional Solution: Recovery receipts and constitutional audit
  Remaining Cost: Recovery overhead
- **Problem:** Incomplete deletion
  Constitutional Solution: Deletion semantics across replicas, indexes, snapshots, and receipts
  Remaining Cost: Tombstone and audit cost

Appendix E: Popular Frame Translation Table

Popular Representation	Constitutional Translation
Filing cabinet	Artifact arena plus descriptor index
Full parking lot	Undergoverned occupancy plus undergoverned absence under high density
Greedy algorithm	Local mutation without long-term health obligation
Traffic jam / cluster	Fragmentation or probe-path congestion
Firebreak	Strategic preserved emptiness
Elastic Hashing	Implicit governance of holes inside lookup geometry
Things table	Explicit governance of object identity and placement
Spaces table	Explicit governance of absence
Defragmentation	Repayment of constitutional debt
Fast lane	Licensed low-overhead jurisdiction for trivial state
99% capacity	Density regime requiring explicit absence governance
Speed limit	Bound produced by a constrained representation
Edge efficiency	Benefit of keeping expensive governance proportional to consequence

---

Appendix F: Lane Taxonomy

Full Constitutional Lane

Used for:

```
durable artifacts
cross-jurisdictional objects
```

```
semantic records
authority-bearing state
regulated workflows
high-value writes
```

Properties:

```
per-operation admission
full receipts
strong replayability
explicit semantic commitments
```

Batched Lane

Used for:

```
many small writes
micro-artifacts
index updates
short bursts
repair summaries
```

Properties:

```
aggregate admission
Merkleized suboperations
batch receipt
individual inclusion proofs
```

Fast Lane

Used for:

```
tiny hot values
transient cache entries
compact indexes
low-risk counters
allocator internals
```

Properties:

```
pre-authorized scope
low metadata
periodic reconciliation
strict escalation rules
```

Repair Lane

Used for:

```
defragmentation
compaction
coalescing
migration
index rebuilding
lane reconciliation
```

Properties:

```
state-preserving movement
repair receipts
bounded cost budgets
health improvement requirements
```

Appendix G: Failure Classes

```
UnderRecordedAbsence:
    Free-space state cannot explain available capacity.

DescriptorDrift:
    Thing descriptor no longer matches stored bytes.

DoubleAllocation:
    Two descriptors claim overlapping extents.

SilentFragmentationDebt:
    Fragmentation exceeds bounds without repair obligation.

UnlicensedFastPath:
    Operation used fast lane without lane authority.

FastLaneEscape:
    Operation exceeded fast-lane scope but did not escalate.

UnreceiptedMove:
    Artifact placement changed without movement receipt.

SemanticMisclassification:
    Meaningful object treated as trivial fast-lane state.

RepairDefault:
    Repair obligation exceeded deadline.

PinnedCompactionFailure:
    Repair plan required moving an immovable object.
```

```
ReceiptStateDivergence:
    Receipt ledger cannot replay current state.

UnderGovernedDeletion:
    Current absence exists without historical explanation.

FalseTriviality:
    Operation claimed fast-lane status despite semantic or authority
    significance.

OverGovernance:
    Operation forced into full governance despite satisfying
    trivial-lane criteria,
    causing avoidable overhead.

UnboundedRepair:
    Repair obligation exists without deadline, budget, or escalation
    rule.
```

Appendix H: Minimal Constitutional Rule Set

A minimal Constitutional Storage constitution should specify:

1. Object descriptor format.
2. Free-space representation.
3. Allocation admission rule.
4. Deletion admission rule.
5. Movement admission rule.
6. Fragmentation metrics.
7. Repair thresholds.
8. Receipt requirements.
9. Lane taxonomy.
10. Fast-lane admission rule.
11. Fast-lane reconciliation rule.
12. Escalation rule.
13. Crash recovery rule.
14. Concurrency ordering rule.
15. Authority model for administrative repair.
16. Occupancy legitimacy rule.
17. Capacity jurisdiction rule.
18. Recovery legitimacy rule.

Without these, the system may still store bytes.

It cannot fully justify state.

Corpus-Wide Reference Positioning

This paper argues that modern storage systems already manage free space through free lists, allocators, compaction, tiering, garbage collection, replication, quotas, snapshots, reserves, and recovery logs. Its nearest external comparison fields are governance, storage, observation. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. Fixity and preservation can keep bytes stable while representation dependencies, executability, interpretation, and institutional custody decay. Telemetry can be complete at the instrumented boundary while omitting the causal variable, real-world effect, or authority question that matters. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E2**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E3**] Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

[**E4**] OECD. *Recommendation of the Council on Artificial Intelligence*. OECD/LEGAL/0449, adopted 2019 and updated 2024. legalinstruments.oecd.org source **Relationship:** Adjacent policy standard for accountability, traceability, transparency, safety, and human-centered values.

[**E5**] Consultative Committee for Space Data Systems. *Reference Model for an Open Archival Information System (OAIS)*. CCSDS 650.0-M-3, Issue 3, December 2024. https://ccsds.org/Pubs/650x0m3.pdf **Relationship:** Predecessor preservation model for information packages, representation information, and designated communities.

[**E6**] John Kunze et al. *The BagIt File Packaging Format (V1.0)*. RFC 8493, 2018. https://www.rfc-editor.org/rfc/rfc8493 **Relationship:** Adjacent custody and transfer format for payload manifests, fixity, and complete package movement.

[**E7**] Juan Benet. "IPFS - Content Addressed, Versioned, P2P File System." 2014. arXiv:1407.3561. https://arxiv.org/abs/1407.3561 **Relationship:** Predecessor content-addressed network architecture; content identity is compared with, not treated as sufficient for, semantic jurisdiction.

Ableman corpus

[**A1**] Adam Ableman. *Ableman Constitutional Governance*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Governance Engineering.* Ableman Corpus current edition; work identity `PAPER.PUB.18749870`;
source SHA-256
`372169F315D192733856B1F4A3F1038BBC73F57CF3ED9ACABAE72B965A69C8D0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Universal Governance Kernel (UGK).* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-1F9BFB2BF8370226`;
source SHA-256
`AA5106EA71E5DF120E141435C280DFC532715CA5DE0C75B8862BB7784ECCFACD`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Legitimacy Without Self-Ratification.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-EF2382F9397335AC`;
source SHA-256
`0186A39C808F8FDDB5C8F42384DF1A26678928434BD50A0EAE8D4947390F68A2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Cold Boot Continuity as a Deployment Primitive.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-3DB3470715D914D0`;
source SHA-256
`4CBB05E90E4ECA37EDA78C0E1FC1FDD180B2D6E0936831B57379AD7124776D4E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Semantic Custody.* Ableman Corpus current edition; work identity `PAPER.PUB.20300952`;
source SHA-256
`E3887FE98E6369E39AC3FC2EAC63F2DF7C2AD18AD682B3A745C0CD0AD241D0F0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Hash Tables and Open Addressing

Bender, Michael A., Kuszmaul, William, and Zhou, Renfei. 2024. "Tight Bounds for Classical Open Addressing." *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS 2024)*, pp. 636-657. https://doi.org/10.1109/FOCS61266.2024.00047. arXiv:2409.11280. Companion result establishing tight bounds for classical open-addressing schemes via the rainbow-hashing construction, relevant to the present paper's treatment of dense open-addressing failure modes.

Farach-Colton, Martin, Krapivin, Andrew, and Kuszmaul, William. 2024. "Optimal Bounds for Open Addressing Without Reordering." *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS 2024)*, pp. 594-605. https://doi.org/10.1109/FOCS61266.2024.00045. arXiv:2501.02305. Result overturning the long-standing conjectural intuition that uniform probing was optimal for open-addressing hash tables, and the motivating impetus for the present paper's treatment of empty space as governed state.

Bender, Michael A., Conway, Alex, Farach-Colton, Martin, Kuszmaul, William, and Tagliavini, Guido. 2023. "Tiny Pointers." *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms (SODA 2023)*, pp. 477-508. https://doi.org/10.1137/1.9781611977554.ch21. Extended version: *ACM Transactions on Algorithms* 21(4), Article 38. https://doi.org/10.1145/3700594. arXiv:2111.12800. Part of the intellectual lineage from compact addressing to dense hash-table design that informs the present paper's treatment of object descriptors as separable from object placement.

Knuth, Donald E. 1998. *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley. Chapter 6.4. Foundational treatment of hashing techniques including open addressing, chaining, and load-factor analysis.

Pagh, Rasmus, and Rodler, Flemming Friche. 2004. "Cuckoo Hashing." *Journal of Algorithms* 51(2): 122-144. https://doi.org/10.1016/j.jalgor.2003.12.002. Alternative collision-resolution strategy using multiple hash functions and constant-time worst-case lookup, relevant to the paper's discussion of separating lookup geometry from space allocation.

Yao, Andrew C. 1985. "Uniform Hashing is Optimal." *Journal of the ACM* 32(3): 687-693. https://doi.org/10.1145/3828.3835. Foundational analysis of hashing under uniform-probing assumptions whose long-standing conjecture about dense open addressing the Farach-Colton, Krapivin, and Kuszmaul (2024) result substantially revises.

Storage Systems, Allocators, and File Systems

Knuth, Donald E. 1997. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley. Section 2.5 on dynamic storage

allocation. Foundational treatment of allocation, fragmentation, and free-list management whose concerns the present paper recasts as constitutional questions over Spaces tables.

Litwin, Witold. 1980. "Linear Hashing: A New Tool for File and Table Addressing." *Proceedings of the 6th International Conference on Very Large Data Bases*: 212-223. Foundational treatment of dynamic hash-table expansion under load, relevant to the paper's discussion of how dense storage failures manifest as representation failures rather than fundamental capacity limits.

McKusick, Marshall Kirk, Joy, William N., Leffler, Samuel J., and Fabry, Robert S. 1984. "A Fast File System for UNIX." *ACM Transactions on Computer Systems* 2(3): 181-197. https://doi.org/10.1145/989.990. Foundational treatment of file-system layout, cylinder groups, and free-space management whose concerns the present paper generalizes to the constitutional layer.

Rosenblum, Mendel, and Ousterhout, John K. 1992. "The Design and Implementation of a Log-Structured File System." *ACM Transactions on Computer Systems* 10(1): 26-52. https://doi.org/10.1145/146941.146943. Treatment of storage as append-only history with derived current state, an architectural pattern that informs the present paper's treatment of receipts as immutable history with derived current authority.

Wilson, Paul R., Johnstone, Mark S., Neely, Michael, and Boles, David. 1995. "Dynamic Storage Allocation: A Survey and Critical Review." *Proceedings of the International Workshop on Memory Management*: 1-116. https://doi.org/10.1007/3-540-60368-9_19. Survey of allocator design choices, fragmentation, and compaction strategies relevant to the paper's discussion of repair governance.

Popular Coverage

Nadis, Steve. 2025. "Undergraduate Upends a 40-Year-Old Data Science Conjecture." *Quanta Magazine*, February 10, 2025. quantamagazine.org source. Popular coverage of the Farach-Colton, Krapivin, and Kuszmaul (2024) open-addressing result, including commentary from Guy Blelloch and Sepehr Assadi on the significance of the breakthrough. The Quanta article provided the motivating presentation that prompted the present paper's reframing of empty space as constitutional state.

Companion Papers in the Constitutional Computing Research Series

Adam Ableman. 2025. "Constitutional Computation: Computing Through Authorized Transformation." Constitutional Computing Research Manuscript. https://doi.org/10.5281/zenodo.20624487

Adam Ableman. 2025. "Constitutional Legends: Jurisdictional Projection Systems for Governed Semantic Participation." Constitutional Computing Research Manuscript. https://doi.org/10.5281/zenodo.20330323

Adam Ableman. In preparation. “The Three Hash Rule: Governance Completeness Through Independent Cryptographic Commitments.” Manuscript.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.20788662